

Oficina 3 — C para Registradores

Tipos, bases numéricas, operadores bit a bit e máscaras

Raoni F. S. Teixeira · Rodolfo Quadros · DENE/UFMT · 1 sessão (2 h) · sem nota · grupos de 3

OBJETIVOS DESTA SESSÃO

Ao final desta sessão o grupo deve ser capaz de:

1. Converter entre binário, hexadecimal e decimal sem calculadora, para valores de oito bits;
2. Explicar o que faz cada um dos operadores `&`, `|`, `^`, `~`, `<<` e `>>`;
3. Construir uma máscara a partir do número do bit e usá-la para ligar, apagar, inverter e testar um bit isolado;
4. Justificar por que se usa `uint8_t` e não `int` em código embarcado;
5. Usar um LED como instrumento de depuração.

POR QUE ESTA SESSÃO NÃO TEM NOTA

Última das três oficinas, e a única cujo conteúdo reaparece integralmente na semana seguinte: o Roteiro 1 avalia exatamente estas operações. Isto aqui é o ensaio, e o ensaio existe para que o erro aconteça aqui.

Aproveite: escreva máscaras erradas de propósito e veja o que sai. Você não terá essa liberdade de novo.

CONFIGURAÇÃO DE BANCADA

Chaves SWITCHS (PORTB)	Todas em OFF
CH2 - 1 (LCD)	OFF
CH3 - 3 e CH3 - 5	OFF
Demais bancos	OFF

Os únicos periféricos usados nesta sessão são os oito LEDs.

1 O que muda em relação a Algoritmos

Em Algoritmos você manipula **valores**: somar, comparar, atribuir. Uma variável é um lugar onde se guarda um número, e escrever nela não tem efeito colateral.

Aqui o programa conversa com **registradores**, e um registrador de porta não é uma variável comum — é um painel de comandos compartilhado, em que cada bit comanda um pino físico diferente.

CONCEITO CENTRAL

`TRISC = 0x00` configura os oito pinos de uma vez, **inclusive os sete que você não queria tocar**. Em um sistema com um atuador ligado a um deles, isso é um acidente, não uma questão de estilo.

`TRISC &= ~0b00000110` é a forma de mexer em dois pinos preservando os outros seis. Entender essa linha é programação de microcontrolador — não é pré-requisito dela. Máscara de bits é o vocabulário nativo da disciplina, e é por isso que ela tem uma sessão inteira antes de valer nota.

2 Bases: binário e hexadecimal

Um registrador de oito bits tem 256 estados. Escrevê-los em decimal esconde exatamente a informação que interessa — quais bits estão em 1.

Binário	Hex	Decimal	Leitura
0b00000001	0x01	1	só o bit 0
0b00001000	0x08	8	só o bit 3
0b10000000	0x80	128	só o bit 7
0b00001111	0x0F	15	os quatro bits baixos
0b11110000	0xF0	240	os quatro bits altos
0b11111111	0xFF	255	todos

Tabela 1: Valores que vale a pena reconhecer de imediato.

Cada dígito hexadecimal corresponde a exatamente quatro bits, o que torna a conversão mecânica: separe o byte em dois grupos de quatro e traduza cada um.

TAREFA

Complete no caderno, sem calculadora:

Binário	Hex	Decimal
0b00100000		
	0x24	
		192
0b01010101		
	0xC3	

Depois, confira com o colega ao lado antes de conferir com a máquina.

3 Tipos: por que uint8_t e não int**⚠ ATENÇÃO**

O tamanho de `int` não é definido pela linguagem C — ele varia entre plataformas. Código escrito com `int` funciona em uma arquitetura e falha silenciosamente em outra.

Em código embarcado usa-se sempre a largura explícita: `uint8_t` para um byte, `uint16_t` para dois, `int16_t` quando o valor pode ser negativo. Isso exige o cabeçalho `<stdint.h>`.

Tipo	Faixa	Uso típico neste curso
<code>uint8_t</code>	0 a 255	Registrador, máscara, contador curto
<code>uint16_t</code>	0 a 65535	Leitura do conversor A/D, preload de temporizador
<code>int16_t</code>	-32768 a 32767	Temperatura em décimos de grau

Tabela 2: Tipos de largura explícita e onde aparecem.

OBSERVAÇÃO

Há uma segunda razão, específica desta plataforma: o PIC18F4550 tem 2 KB de RAM. Um `int` de 16 bits onde bastariam 8 desperdiça metade do espaço, e o desperdício não aparece até o dia em que o programa deixa de caber — momento em que já é caro de corrigir.

E uma terceira: o compilador XC8 na versão gratuita não gera código de ponto flutuante utilizável neste curso. Por isso a temperatura será armazenada em **décimos de grau** como `int16_t`, e não como `float`. Restrição da ferramenta virando decisão de projeto — padrão que vai se repetir.

4 Os operadores

Operador	Nome	Para que serve, na prática
<code>&</code>	E bit a bit	Zerar bits; testar se um bit está em 1
<code> </code>	OU bit a bit	Colocar bits em 1 sem tocar nos demais
<code>^</code>	OU-exclusivo	Inverter bits selecionados
<code>~</code>	Complemento	Transformar uma máscara em seu negativo
<code><<</code>	Deslocamento à esquerda	Construir a máscara do bit n
<code>>></code>	Deslocamento à direita	Trazer um campo para os bits baixos

Tabela 3: Os seis operadores e sua função de projeto.

⚠ ATENÇÃO

Não confunda `&` com `&&`, nem `|` com `||`. Os dobrados são operadores **lógicos**: trabalham com verdadeiro e falso e devolvem 0 ou 1. Os simples trabalham bit a bit.

`PORTB && 0x04` quase nunca é o que você quis escrever, e o compilador não reclama.

4.1 As quatro operações fundamentais

```
#include <stdint.h>

#define BIT(n) (1u << (n))

uint8_t r = 0b00000000;

r |= BIT(2);           /* liga o bit 2, preserva o resto */
r &= ~BIT(2);          /* apaga o bit 2, preserva o resto */
r ^= BIT(2);           /* inverte o bit 2 */

if (r & BIT(2)) {       /* testa: verdadeiro se o bit 2 é 1 */
    /* ... */
}
```

CONCEITO CENTRAL

As quatro linhas acima resolvem praticamente todo o acesso a registrador deste semestre. Vale decorá-las — não como fórmula, mas reconhecendo o padrão: **ligar é OU com a máscara; apagar é E com a máscara invertida; inverter é OU-exclusivo; testar é E seguido de comparação.**

O `~` no apagar é o erro mais comum de todos. Sem ele, `r &= BIT(2)` zera todos os outros bits em vez de preservar — e o sintoma, “mexi em um e sumiram os outros”, é inconfundível quando você já o viu uma vez.

TAREFA

Complete a tabela no caderno, escrevendo binário e hexadecimal:

Expressão	Binário	Hex
BIT(0)		
BIT(3)		
BIT(7)		
BIT(2) BIT(5)		
~BIT(0)		
~(BIT(1) BIT(6))		

TAREFA

Partindo de `uint8_t r = 0b10110100;`, escreva o valor de `r` após cada linha, **em sequência** — cada uma opera sobre o resultado da anterior:

1. `r |= BIT(0);`
2. `r &= ~BIT(4);`
3. `r ^= 0b00001111;`
4. `r = BIT(1);`

A quarta linha é a pegadinha. Explique em uma frase por que ela é diferente das três anteriores.

5 O LED como instrumento

Você ainda não tem display, nem serial, nem depurador. Tem oito LEDs — e eles bastam para muito mais do que parece.

Pergunta	Como o LED responde
O programa chegou até aqui?	Acender um LED na linha suspeita
Este <code>if</code> está entrando?	Acender dentro do bloco, apagar fora
Qual o valor deste byte?	Escrever o byte inteiro na porta dos LEDs
Com que frequência isso roda?	Inverter um LED a cada passagem e observar

Tabela 4: Depuração com o que existe na bancada.

OBSERVAÇÃO

A quarta linha é a mais poderosa e a menos óbvia: um LED que inverte a cada passagem por um trecho de código transforma frequência de execução em algo visível. Se ele pisca rápido demais para o olho, o osciloscópio lê o período — que foi exatamente a medida usada na Oficina 2 para descobrir o clock real.

TAREFA

Escreva, compile e grave um programa que escreva um valor fixo na porta dos LEDs, usando TRIS e LAT na ordem correta — a que você discutiu na Oficina

2. Comece com `0b11110000`.
3. Desenhe os oito círculos no caderno e marque quais LEDs você **espera** que acendam.
4. Grave e compare com o que aconteceu.

EXPERIMENTE E ANOTE

O resultado provavelmente não foi o que você previu.

Não tente explicar agora, e não peça a explicação. Registre com precisão: qual byte você escreveu e quais LEDs acenderam. Formule uma hipótese sobre o circuito — não sobre o seu código — que explique a diferença.

O Roteiro 1, na semana que vem, começa exatamente por aí. Chegar lá com a hipótese já escrita muda completamente o que aquela sessão vale para você.

6 Erros de compilação que você vai encontrar

Mensagem	O que costuma ser
undeclared identifier 'uint8_t'	Faltou <code>#include <stdint.h></code>
expected ';'	O ponto-e-vírgula que falta está na linha anterior
implicit declaration of function	Função usada antes de declarada, ou cabeçalho ausente
'LATD' undeclared	Faltou <code>#include <xc.h></code> , ou o dispositivo do projeto está errado
Compila mas nada muda	Gravou um .hex antigo — ver Oficina 2

Tabela 5: Primeiros erros do compilador, traduzidos.

OBSERVAÇÃO

Leia sempre a **primeira** mensagem de erro, não a última. O compilador perde o fio depois do primeiro tropeço e passa a reclamar de linhas que estão perfeitas. Corrigir a primeira costuma apagar dez.

7 Fechamento

TAREFA

Devolva a bancada ao estado inicial e leve o caderno preenchido.

TAREFA

Para o Roteiro 1. Traga escrito: no kit XM118, RC1 aciona a resistência de aquecimento e RC2 aciona a ventoinha; RC6 e RC7 são a comunicação serial.

Um colega precisa ligar a ventoinha e escreve `LATC = 0b00000100;`. O que acontece com a comunicação serial? E com a resistência?

A partir da semana que vem, a previsão registrada antes do experimento vale nota — e é avaliada pelo raciocínio, não pelo acerto.